OOP stands for **Object-Oriented Programming**.

Python is an object-oriented language, allowing you to structure your code using classes and objects for better organization and reusability.

Advantages of OOP

- Provides a clear structure to programs
- Makes code easier to maintain, reuse, and debug
- Helps keep your code DRY (**Don't Repeat Yourself**)
- Allows you to build reusable applications with less code

Tip: **The DRY principle means you should avoid writing the same code more than once. Move repeated code into functions or classes and reuse it.**

What are Classes and Objects?

Classes and objects are the two core concepts in object-oriented programming.

A class defines what an object should look like, and an object is created based on that class.

For example:

Class	Object
Fruit	Apple, Banana, Mango
Car	Volvo, Audi, Toyota

When you create an object from a class, it inherits all the variables and functions defined inside that class.

Python Classes/Objects

Python is an object-oriented programming language.

Almost everything in Python is an object, with its properties and methods.

A Class is like an object constructor, or a "**blueprint**" for creating objects.

Create a Class

To create a class, use the keyword **class:**

Example:
Create a class named MyClass, with a property named x:

```
class MyClass:
  x = 5
```

Create Object

Now we can use the class named MyClass to create objects:

Example:
Create an object named p1, and print the value of x:

```
p1 = MyClass()
print(p1.x)
```

Delete Objects

You can delete objects by using the **del** keyword:

Example:
Delete the p1 object:

```
del p1
```

Multiple Objects

You can create multiple objects from the same class

Example:
Create three objects from the MyClass class:

```
p1 = MyClass()
p2 = MyClass()
p3 = MyClass()

print(p1.x)
print(p2.x)
print(p3.x)
```

Note: Each object is independent and has its own copy of the class properties.

The pass Statement

class definitions cannot be empty, but if you for some reason have a **class** definition with no content, put in the **pass** statement to avoid getting an error.

Example:

```
class Person:
  pass
```

The __init__() Method

All classes have a built-in method called `__init__()`, which is always executed when the class is being initiated.

The `__init__()` method is used to assign values to object properties, or to perform operations that are necessary when the object is being created.

Example:

Create a class named Person, use the `__init__()` method to assign values for name and age:

```
class Person:
  def __init__(self, name, age):
    self.name = name
    self.age = age

p1 = Person("Emil", 36)

print(p1.name)
print(p1.age)
```

Note: The `__init__()` method is called automatically every time the class is being used to create a new object.

Why Use __init__()?

Without the `__init__()` method, you would need to set properties manually for each object:

Example:

Create a class without `__init__()`:

```
class Person:
  pass
```

```
p1 = Person()
p1.name = "Tobias"
p1.age = 25

print(p1.name)
print(p1.age)
```

Using `__init__()` makes it easier to create objects with initial values:

Example

With `__init__()`, you can set initial values when creating the object:

```
class Person:
  def __init__(self, name, age):
    self.name = name
    self.age = age

p1 = Person("Linus", 28)

print(p1.name)
print(p1.age)
```

Default Values in __init__()

You can also set default values for parameters in the `__init__()` method:

Example:

Set a default value for the age parameter:

```
class Person:
  def __init__(self, name, age=18):
    self.name = name
    self.age = age

p1 = Person("Emil")
p2 = Person("Tobias", 25)

print(p1.name, p1.age)
print(p2.name, p2.age)
```

Multiple Parameters

The `__init__()` method can have as many parameters as you need:

Example:

Create a Person class with multiple parameters:

```
class Person:
  def __init__(self, name, age, city, country):
    self.name = name
    self.age = age
    self.city = city
    self.country = country

p1 = Person("Linus", 30, "Oslo", "Norway")

print(p1.name)
print(p1.age)
print(p1.city)
print(p1.country)
```

The `self` Parameter

The `self` parameter is a reference to the current instance of the class.

It is used to access properties and methods that belong to the class.

Example:

Use `self` to access class properties:

```
class Person:
  def __init__(self, name, age):
    self.name = name
    self.age = age

  def greet(self):
    print("Hello, my name is " + self.name)

p1 = Person("Emil", 25)
p1.greet()
```

Note: The `self` parameter must be the first parameter of any method in the class.

Why Use self?

Without `self`, Python would not know which object's properties you want to access:

Example:

The `self` parameter links the method to the specific object:

```
class Person:
  def __init__(self, name):
    self.name = name

  def printname(self):
    print(self.name)

p1 = Person("Tobias")
p2 = Person("Linus")

p1.printname()
p2.printname()
```

self Does Not Have to Be Named "self"

It does not have to be named `self`, you can call it whatever you like, but it has to be the **first parameter** of any method in the class:

Example:

Use the words *myobject* and *abc* instead of *self*:

```
class Person:
  def __init__(myobject, name, age):
    myobject.name = name
    myobject.age = age

  def greet(abc):
    print("Hello, my name is " + abc.name)

p1 = Person("Emil", 36)
p1.greet()
```

Note: While you *can* use a different name, it is strongly recommended to use `self` as it is the convention in Python and makes your code more readable to others.

Accessing Properties with self

You can access any property of the class using `self`:

Example:

Access multiple properties using `self`:

```
class Car:
  def __init__(self, brand, model, year):
    self.brand = brand
    self.model = model
    self.year = year

  def display_info(self):
    print(f"{self.year} {self.brand} {self.model}")

car1 = Car("Toyota", "Corolla", 2020)
car1.display_info()
```

Calling Methods with self

You can also call other methods within the class using `self`:

Example:

Call one method from another method using `self`:

```
class Person:
  def __init__(self, name):
    self.name = name

  def greet(self):
    return "Hello, " + self.name

  def welcome(self):
    message = self.greet()
    print(message + "! Welcome to our website.")

p1 = Person("Tobias")
p1.welcome()
```

Python Class Properties

Class Properties

Properties are variables that belong to a class. They store data for each object created from the class.

Example:

Create a class with properties:

```
class Person:
  def __init__(self, name, age):
    self.name = name
    self.age = age

p1 = Person("Emil", 36)

print(p1.name)
print(p1.age)
```

Access Properties

You can access object properties using dot notation:

Example:

Access the properties of an object:

```
class Car:
  def __init__(self, brand, model):
    self.brand = brand
    self.model = model

car1 = Car("Toyota", "Corolla")

print(car1.brand)
print(car1.model)
```

Modify Properties

You can modify the value of properties on objects:

Example:

Change the age property:

```
Ac class Person:
  def __init__(self, name, age):
    self.name = name
    self.age = age

p1 = Person("Tobias", 25)
print(p1.age)

p1.age = 26
print(p1.age)
```

Delete Properties

You can delete properties from objects using the `del` keyword:

Example:

Delete the age property:

```
class Person:
  def __init__(self, name, age):
    self.name = name
    self.age = age

p1 = Person("Linus", 30)

del p1.age

print(p1.name) # This works
# print(p1.age) # This would cause an error
```

Class Properties vs Object Properties

Properties defined inside `__init__()` belong to each object (instance properties).

Properties defined outside methods belong to the class itself (class properties) and are shared by all objects:

Example:

Class property vs instance property:

```
class Person:
  species = "Human" # Class property

  def __init__(self, name):
    self.name = name # Instance property

p1 = Person("Emil")
p2 = Person("Tobias")

print(p1.name)
print(p2.name)
print(p1.species)
print(p2.species)
```

Modifying Class Properties

When you modify a class property, it affects all objects:

Example:

Change the class property:

```
class Person:
  lastname = ""

  def __init__(self, name):
    self.name = name

p1 = Person("Linus")
p2 = Person("Emil")

Person.lastname = "Refsnes"

print(p1.lastname)
print(p2.lastname)
```

Add New Properties

You can add new properties to existing objects:

Example:

Add a new property to an object:

```
class Person:
  def __init__(self, name):
    self.name = name

p1 = Person("Tobias")

p1.age = 25
p1.city = "Oslo"

print(p1.name)
print(p1.age)
print(p1.city)
```

Note: Adding properties this way only adds them to that specific object, not to all objects of the class.

Python Class Methods

Class Methods

Methods are functions that belong to a class. They define the behaviour of objects created from the class.

Example:

Create a method in a class:

```
class Person:
  def __init__(self, name):
    self.name = name

  def greet(self):
    print("Hello, my name is " + self.name)

p1 = Person("Emil")
p1.greet()
```

Note: All methods must have `self` as the first parameter.

Methods with Parameters

Methods can accept parameters just like regular functions:

Example:

Create a method with parameters:

```
class Calculator:
  def add(self, a, b):
    return a + b

  def multiply(self, a, b):
    return a * b

calc = Calculator()
print(calc.add(5, 3))
print(calc.multiply(4, 7))
```

Methods Accessing Properties

Methods can access and modify object properties using `self`:

Example:

A method that accesses object properties:

```
class Person:
  def __init__(self, name, age):
    self.name = name
    self.age = age

  def get_info(self):
    return f"{self.name} is {self.age} years old"

p1 = Person("Tobias", 28)
print(p1.get_info())
```

Methods Modifying Properties

Methods can modify the properties of an object:

Example:

A method that changes a property value:

```
class Person:
  def __init__(self, name, age):
    self.name = name
    self.age = age

  def celebrate_birthday(self):
    self.age += 1
    print(f"Happy birthday! You are now {self.age}")

p1 = Person("Linus", 25)
p1.celebrate_birthday()
p1.celebrate_birthday()
```

The __str__() Method

The __str__() method is a special method that controls what is returned when the object is printed:

Example:

Without the __str__() method:

```
class Person:
  def __init__(self, name, age):
    self.name = name
    self.age = age

p1 = Person("Emil", 36)
print(p1)
```

Example:

With the __str__() method:

```
class Person:
  def __init__(self, name, age):
    self.name = name
    self.age = age

  def __str__(self):
    return f"{self.name} ({self.age})"

p1 = Person("Tobias", 36)
print(p1)
```

Multiple Methods

A class can have multiple methods that work together:

Example:

Create multiple methods in a class:

```
class Playlist:
  def __init__(self, name):
    self.name = name
    self.songs = []

  def add_song(self, song):
    self.songs.append(song)
    print(f"Added: {song}")

  def remove_song(self, song):
    if song in self.songs:
      self.songs.remove(song)
      print(f"Removed: {song}")

  def show_songs(self):
    print(f"Playlist '{self.name}':")
    for song in self.songs:
      print(f"- {song}")

my_playlist = Playlist("Favorites")
my_playlist.add_song("Bohemian Rhapsody")
my_playlist.add_song("Stairway to Heaven")
my_playlist.show_songs()
```

Delete Methods

You can delete methods from a class using the `del` keyword:

Example:

Delete a method from a class:

```
class Person:
  def __init__(self, name):
    self.name = name

  def greet(self):
    print("Hello!")

p1 = Person("Emil")

del Person.greet

p1.greet() # This will cause an error
```

Python Inheritance

Inheritance allows us to define a class that inherits all the methods and properties from another class.

Parent class is the class being inherited from, also called base class.

Child class is the class that inherits from another class, also called derived class.

Create a Parent class

Any class can be a parent class, so the syntax is the same as creating any other class:

Example:

Create a class named Person, with firstname and lastname properties, and a printname method:

```
class Person:
  def __init__(self, fname, lname):
    self.firstname = fname
    self.lastname = lname

  def printname(self):
    print(self.firstname, self.lastname)

#Use the Person class to create an object, and then execute the printname method:

x = Person("John", "Doe")
x.printname()
```

Create a Child Class

To create a class that inherits the functionality from another class, send the parent class as a parameter when creating the child class:

Example:

Create a class named `Student`, which will inherit the properties and methods from the `Person` class:

```
class Student(Person):
  pass
```

Note: Use the `pass` keyword when you do not want to add any other properties or methods to the class.

Now the Student class has the same properties and methods as the Person class.

Example:

Use the `Student` class to create an object, and then execute the `printname` method:

```
class Person:
  def __init__(self, fname, lname):
    self.firstname = fname
    self.lastname = lname

  def printname(self):
    print(self.firstname, self.lastname)

class Student(Person):
  pass

x = Student("Mike", "Olsen")
x.printname()
```

Add the __init__() Function

So far we have created a child class that inherits the properties and methods from its parent.

We want to add the `__init__()` function to the child class (instead of the `pass` keyword).

Note: The `__init__()` function is called automatically every time the class is being used to create a new object.

Example:

Add the `__init__()` function to the `Student` class:

```
class Student(Person):
  def __init__(self, fname, lname):
    #add properties etc.
```

To keep the inheritance of the parent's `__init__()` function, add a call to the parent's `__init__()` function:

Example:

```
class Person:
  def __init__(self, fname, lname):
    self.firstname = fname
    self.lastname = lname

  def printname(self):
    print(self.firstname, self.lastname)

class Student(Person):
  def __init__(self, fname, lname):
    Person.__init__(self, fname, lname)

x = Student("Mike", "Olsen")
x.printname()
```

Use the super() Function

Python also has a `super()` function that will make the child class inherit all the methods and properties from its parent:

```
class Person:
  def __init__(self, fname, lname):
    self.firstname = fname
    self.lastname = lname

  def printname(self):
    print(self.firstname, self.lastname)

class Student(Person):
  def __init__(self, fname, lname):
    super().__init__(fname, lname)

x = Student("Mike", "Olsen")
x.printname()
```

By using the `super()` function, you do not have to use the name of the parent element, it will automatically inherit the methods and properties from its parent.

Add Properties

Example:

Add a property called `graduationyear` to the `Student` class:

```
class Person:
  def __init__(self, fname, lname):
    self.firstname = fname
    self.lastname = lname

  def printname(self):
    print(self.firstname, self.lastname)

class Student(Person):
  def __init__(self, fname, lname):
    super().__init__(fname, lname)
    self.graduationyear = 2019

x = Student("Mike", "Olsen")
print(x.graduationyear)
```

In the example below, the year `2019` should be a variable, and passed into the `Student` class when creating student objects. To do so, add another parameter in the `__init__()` function:

Example:

Add a `year` parameter, and pass the correct year when creating objects:

```
class Person:
  def __init__(self, fname, lname):
    self.firstname = fname
    self.lastname = lname

  def printname(self):
    print(self.firstname, self.lastname)

class Student(Person):
  def __init__(self, fname, lname, year):
    super().__init__(fname, lname)
    self.graduationyear = year

x = Student("Mike", "Olsen", 2019)
print(x.graduationyear)
```

Add Methods

Example:

Add a method called `welcome` to the `Student` class:

```
class Person:
  def __init__(self, fname, lname):
    self.firstname = fname
    self.lastname = lname

  def printname(self):
    print(self.firstname, self.lastname)

class Student(Person):
  def __init__(self, fname, lname, year):
    super().__init__(fname, lname)
    self.graduationyear = year

  def welcome(self):
    print("Welcome", self.firstname, self.lastname, "to the class of",
self.graduationyear)

x = Student("Mike", "Olsen", 2024)
x.welcome()
```

If you add a method in the child class with the same name as a function in the parent class, the inheritance of the parent method will be overridden.

Python Polymorphism

The word "polymorphism" means "many forms", and in programming it refers to methods/functions/operators with the same name that can be executed on many objects or classes.

Function Polymorphism

An example of a Python function that can be used on different objects is the `len()` function.

String

For strings `len()` returns the number of characters:

Example:

```
x = "Hello World!"

print(len(x))
```

Tuple

For tuples `len()` returns the number of items in the tuple:

Example:

```
mytuple = ("apple", "banana", "cherry")

print(len(mytuple))
```

Dictionary

For dictionaries `len()` returns the number of key/value pairs in the dictionary:

Example:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}

print(len(thisdict))
```

Class Polymorphism

Polymorphism is often used in Class methods, where we can have multiple classes with the same method name.

For example, say we have three classes: `Car`, `Boat`, and `Plane`, and they all have a method called `move()`:

Example:

Different classes with the same method:

```
class Car:
  def __init__(self, brand, model):
    self.brand = brand
    self.model = model

  defmove(self):
    print("Drive!")

class Boat:
  def __init__(self, brand, model):
    self.brand = brand
    self.model = model

  defmove(self):
    print("Sail!")

class Plane:
  def __init__(self, brand, model):
    self.brand = brand
    self.model = model

  defmove(self):
    print("Fly!")

car1 = Car("Ford", "Mustang")       #Create a Car object
boat1 = Boat("Ibiza", "Touring 20") #Create a Boat object
plane1 = Plane("Boeing", "747")     #Create a Plane object

for x in (car1, boat1, plane1):
  x.move()
```

Look at the for loop at the end. Because of polymorphism we can execute the same method for all three classes.

Inheritance Class Polymorphism

What about classes with child classes with the same name? Can we use polymorphism there?

Yes. If we use the example above and make a parent class called `Vehicle`, and make `Car`, `Boat`, `Plane` child classes of `Vehicle`, the child classes inherits the `Vehicle` methods, but can override them:

Example:

Create a class called `Vehicle` and make `Car`, `Boat`, `Plane` child classes of `Vehicle`:

```
class Vehicle:
  def __init__(self, brand, model):
    self.brand = brand
    self.model = model

  def move(self):
    print("Move!")

class Car(Vehicle):
  pass

class Boat(Vehicle):
  def move(self):
    print("Sail!")

class Plane(Vehicle):
  def move(self):
    print("Fly!")

car1 = Car("Ford", "Mustang")       #Create a Car object
boat1 = Boat("Ibiza", "Touring 20") #Create a Boat object
plane1 = Plane("Boeing", "747")     #Create a Plane object

for x in (car1, boat1, plane1):
  print(x.brand)
  print(x.model)
  x.move()
```

Child classes inherits the properties and methods from the parent class.

In the example above, you can see that the `Car` class is empty, but it inherits `brand`, `model`, and `move()` from `Vehicle`.

The `Boat` and `Plane` classes also inherit `brand`, `model`, and `move()` from `Vehicle`, but they both override the `move()` method.

Because of polymorphism we can execute the same method for all classes.

Python Encapsulation

Encapsulation is about protecting data inside a class.

It means keeping data (properties) and methods together in a class, while controlling how the data can be accessed from outside the class.

This prevents accidental changes to your data and hides the internal details of how your class works.

Private Properties

In Python, you can make properties private by using a double underscore __ prefix:

Example:

Create a private class property named __age:

```
class Person:
  def __init__(self, name, age):
    self.name = name
    self.__age = age # Private property

p1 = Person("Emil", 25)
print(p1.name)
print(p1.__age) # This will cause an error
```

Note: Private properties cannot be accessed directly from outside the class.

Get Private Property Value

To access a private property, you can create a getter method:

Example:

Use a getter method to access a private property:

```
class Person:
  def __init__(self, name, age):
    self.name = name
    self.__age = age

  def get_age(self):
    return self.__age

p1 = Person("Tobias", 25)
print(p1.get_age())
```

Set Private Property Value

To modify a private property, you can create a setter method.

The setter method can also validate the value before setting it:

Example:

Use a setter method to change a private property:

```
class Person:
  def __init__(self, name, age):
    self.name = name
    self.__age = age

  def get_age(self):
    return self.__age

  def set_age(self, age):
    if age > 0:
      self.__age = age
    else:
      print("Age must be positive")

p1 = Person("Tobias", 25)
print(p1.get_age())

p1.set_age(26)
print(p1.get_age())
```

Why Use Encapsulation?

Encapsulation provides several benefits:

- **Data Protection:** Prevents accidental modification of data
- **Validation:** You can validate data before setting it
- **Flexibility:** Internal implementation can change without affecting external code
- **Control:** You have full control over how data is accessed and modified

Example:

Use encapsulation to protect and validate data:

```
class Student:
  def __init__(self, name):
    self.name = name
    self.__grade = 0

  def set_grade(self, grade):
    if 0 <= grade <= 100:
      self.__grade = grade
    else:
      print("Grade must be between 0 and 100")

  def get_grade(self):
    return self.__grade

  def get_status(self):
    if self.__grade>= 60:
      return "Passed"
    else:
      return "Failed"

student = Student("Emil")
student.set_grade(85)
print(student.get_grade())
print(student.get_status())
```

Protected Properties

Python also has a convention for protected properties using a single underscore _ prefix:

Example:

Create a protected property:

```
class Person:
  def __init__(self, name, salary):
    self.name = name
    self._salary = salary # Protected property

p1 = Person("Linus", 50000)
print(p1.name)
print(p1._salary) # Can access, but shouldn't
```

Note: A single underscore _ is just a convention. It tells other programmers that the property is intended for internal use, but Python doesn't enforce this restriction.

Private Methods

You can also make methods private using the double underscore prefix:

Example:

Create a private method:

```
class Calculator:
  def __init__(self):
    self.result = 0

  def __validate(self, num):
    if not isinstance(num, (int, float)):
      return False
    return True

  def add(self, num):
    if self.__validate(num):
      self.result += num
    else:
      print("Invalid number")

calc = Calculator()
calc.add(10)
calc.add(5)
print(calc.result)
# calc.__validate(5) # This would cause an error
```

Note: Just like private properties with double underscores, private methods cannot be called directly from outside the class. The `__validate` method can only be used by other methods inside the class.

Name Mangling

Name mangling is how Python implements private properties and methods.

When you use double underscores `__`, Python automatically renames it internally by adding `_ClassName` in front.

For example, `__age` becomes `_Person__age`.

Example:

See how Python mangles the name:

```
class Person:
  def __init__(self, name, age):
    self.name = name
    self.__age = age

p1 = Person("Emil", 30)

# This is how Python mangles the name:
print(p1._Person__age) # Not recommended!
```

Note: While you *can* access private properties using the mangled name, it's not recommended. It defeats the purpose of encapsulation.

Python Inner Classes

An inner class is a class defined inside another class. The inner class can access the properties and methods of the outer class.

Inner classes are useful for grouping classes that are only used in one place, making your code more organized.

Example:

Create an inner class:

```
class Outer:
  def __init__(self):
    self.name = "Outer Class"

  class Inner:
    def __init__(self):
      self.name = "Inner Class"

    def display(self):
      print("This is the inner class")

outer = Outer()
print(outer.name)
```

Accessing Inner Class from the Outside

To access the inner class, create an object of the outer class, and then create an object of the inner class:

Example:

Access the inner class and create an object:

```
class Outer:
  def __init__(self):
    self.name = "Outer"

  class Inner:
    def __init__(self):
      self.name = "Inner"

    def display(self):
      print("Hello from inner class")

outer = Outer()
inner = outer.Inner()
inner.display()
```

Accessing Outer Class from Inner Class

Inner classes in Python do not automatically have access to the outer class instance.

If you want the inner class to access the outer class, you need to pass the outer class instance as a parameter:

Example:

Pass the outer class instance to the inner class:

```
class Outer:
  def __init__(self):
    self.name = "Emil"

  class Inner:
    def __init__(self, outer):
      self.outer = outer

    def display(self):
      print(f"Outer class name: {self.outer.name}")

outer = Outer()
inner = outer.Inner(outer)
inner.display()
```

Practical Example

Inner classes are useful for creating helper classes that are only used within the context of the outer class:

Use an inner class to represent a car's engine:

```
class Car:
  def __init__(self, brand, model):
    self.brand = brand
    self.model = model
    self.engine = self.Engine()

  class Engine:
    def __init__(self):
      self.status = "Off"

    def start(self):
      self.status = "Running"
      print("Engine started")

    def stop(self):
      self.status = "Off"
      print("Engine stopped")

  def drive(self):
    if self.engine.status == "Running":
      print(f"Driving the {self.brand} {self.model}")
    else:
      print("Start the engine first!")

car = Car("Toyota", "Corolla")
car.drive()
car.engine.start()
car.drive()
```

Multiple Inner Classes

A class can have multiple inner classes:

Example:

Create multiple inner classes:

```
class Computer:
  def __init__(self):
    self.cpu = self.CPU()
    self.ram = self.RAM()

  class CPU:
    def process(self):
      print("Processing data...")

  class RAM:
    def store(self):
      print("Storing data...")

computer = Computer()
computer.cpu.process()
computer.ram.store()
```